Codebase Evolution Intelligence System

Comprehensive Project Specification, Technical Architecture & Interview Defense Guide

Domain: Software Engineering Analytics & Mining	**Target Stack:** Python, MySQL, Streamlit, GitPython, Radon
Target Audience: Tech Interviewers, Recruiters, Evaluators	**Status:** Production Ready & End-to-End Verified

1. Problem Statement

Modern software engineering repositories accrue thousands of commits, hundreds of modified files, and multi-author interactions over years of active development. However, **Git history remains an unmined black box** for engineering leads and developers. Critical operational questions cannot be answered using standard Git commands alone:

- **Invisible Technical Debt:** Which files undergo erratic modification and code churn without delivering proportional functional value?
- **Defect Clustering:** Where do defects and bug fixes historically recur across releases?
- **Complexity Accumulation:** Which files combine complex branching logic with frequent edits, creating high risk for regressions?
- **Knowledge Silos & Bus Factor:** Is critical domain knowledge concentrated in single authors, and how is code ownership distributed?
- **Velocity & Lifecycle Blindness:** Is the codebase expanding steadily, or has it entered a phase of high maintenance churn?

2. The Solution

The **Codebase Evolution Intelligence System** is an automated software repository intelligence platform. Given any public Git/GitHub URL, the system clones the repository, traverses its complete commit DAG, computes static McCabe Cyclomatic Complexity using **Radon**, executes a multi-factor **Hotspot Scoring Algorithm**, persists structured records into a relational **MySQL** database, and delivers high-contrast, interactive visual analytics through a **Streamlit** web dashboard.

> ***Core Value Proposition:*** *Transforms raw, chronological Git commits into actionable, multidimensional engineering insights—enabling proactive refactoring, bug hotspot isolation, and architectural risk mitigation.*

3. Core Features & Capabilities

Feature Module	Technical Functionality & Impact
1. Automated Ingestion & Cloning	Autonomous shallow/full cloning of any public Git/GitHub URL via GitPython with real-time percentage progress streaming.
2. Commit Analytics & Velocity	Parses commit hashes, author metadata, messages, timestamps, files touched, additions (+), and deletions (-). Provides interactive search and date filtering.
3. File Churn & Inventory	Catalogs all repository files, mapping commit frequency, lines of code (LOC), source lines of code (SLOC), and comments across active and deleted file states.
4. Radon Code Complexity	Static AST parsing with Radon to measure McCabe Cyclomatic Complexity (CC) and grade Python modules from Rank A (low risk) to Rank F (extreme risk) without arbitrary execution.
5. Heuristic Bug-Fix Detection	Identifies bug-fixing commits via targeted token heuristics (fix, bug, error, issue, hotfix) and flags them transparently with heuristic attribution.
6. Multi-Factor Code Hotspots	Composite algorithm combining Change Frequency (40%), Bug Involvement (35%), and Radon Complexity (25%) into a 0–100 score with risk classification and actionable refactoring guidance.
7. Contributor Intelligence	Computes developer commit rankings, distinct files touched, net code contributions, and commit share percentages to analyze knowledge distribution and bus factor.
8. Codebase Evolution Visuals	High-contrast Plotly time-series charts for commit velocity, code churn dynamic (+/- lines), net LOC growth curves, bug-fix frequency, and active contributors over time.

4. System Architecture & Working Flow

The architecture is engineered around a clean, layered pipeline separating external Git ingestion, static metric computation, relational persistence, and presentation:

Layer	Component & Execution Responsibility
Presentation Layer	**Streamlit (Multi-page):** Renders 6 analytical views (Overview, Commits, Files, Hotspots, Contributors, Evolution) and interactive Plotly charts styled with high contrast.
Orchestration Layer	**analysis.py:** Coordinates pipeline steps, validates URLs, initiates clone jobs, computes hotspot scores, and structures Pandas DataFrames for dashboard consumption.
Git Ingestion Layer	**git_analyzer.py:** Interacts with local/remote repos via GitPython. Extracts commit logs, file modification diff stats, developer identities, and HEAD tree blobs.

Code Metrics Layer	**metrics.py:** Analyzes Python ASTs via Radon for Cyclomatic Complexity and raw LOC/SLOC. Implements normalized multi-factor hotspot scoring formula.
Persistence Layer	**database.py & models.py:** SQLAlchemy ORM connected to MySQL 8.0 via PyMySQL. Handles automated DB creation, pooled sessions, indexes, and cascading deletes.

End-to-End Execution Flow:

Step 1: URL Input & Validation: The user inputs a public GitHub URL on the Streamlit sidebar. Regex normalizes the URL and extracts repository identity.

Step 2: Autonomous Git Clone / Fetch: GitPython clones the repo into a structured local directory (`./cloned_repos`), streaming percentage progress via a callback.

Step 3: History & Diff Extraction: The engine traverses commits on the active branch, extracting metadata, insertions, deletions, and touched file paths.

Step 4: Radon Complexity Analysis: Active Python files are statically parsed via Radon AST visitors, extracting McCabe Cyclomatic Complexity and rank (A–F).

Step 5: Hotspot Calculation: Each file is scored across normalized change frequency, bug involvement, and complexity.

Step 6: MySQL Relational Persistence: Data is atomically written across 6 relational tables using foreign keys, batch inserts, and cascaded relationships.

Step 7: Dashboard Visualization: Streamlit renders KPIs, search filters, and Plotly charts.

5. Relational Database Schema (MySQL 8.0)

The system persists repository data in a strictly normalized relational schema with 6 tables, foreign key constraints, and performance indexes:

Table	Primary Key & Foreign Keys	Key Columns & Purpose
repositories	PK: id	name, url, default_branch, cloned_path, total_commits, total_files, total_loc, analyzed_at.
developers	PK: id FK: repository_id	name, email, total_commits, total_lines_added, total_lines_deleted, first_commit_date, last_commit_date.
commits	PK: id FK: repository_id, developer_id	commit_hash, author_name, author_email, message, commit_date, files_changed, lines_added, lines_deleted, is_bug_fix.
files	PK: id FK: repository_id	file_path (utf8mb4_bin), file_name, file_extension, current_loc, change_count, lines_added, lines_deleted, bug_fix_count, is_active.
commit_files	PK: id FK: commit_id, file_id	Associative junction table storing change_type ('M', 'A', 'D'), lines_added, lines_deleted per commit-file touch.
file_metrics	PK: id FK: file_id (Unique)	loc, sloc, comments, cyclomatic_complexity, complexity_rank ('A'–'F'), hotspot_score, analyzed_at.

6. Technology Stack & Justification

Technology	Role in System	Why Chosen Over Alternatives
Python 3.12	Core programming language	Rich static analysis ecosystem (AST parsing, Git bindings, data science libraries).
Streamlit	User interface & dashboard	Rapid interactive multi-page web applications in pure Python without heavy JS frontend overhead.
MySQL 8.0 & SQLAlchemy	Relational database & ORM	ACID compliance, strict relational constraints (cascading deletes), indexing, connection pooling, and structured querying.
GitPython	Git repository interface	Direct object-level inspection of Git repositories, commit DAG traversal, tree blobs, and remote progress reporting.
Radon	Static code complexity engine	Computes McCabe Cyclomatic Complexity by parsing Python ASTs statically—no code execution required.
Pandas	Data manipulation & aggregation	Vectorized operations, rolling window calculations, and time-series resampling (daily, weekly, monthly).
Plotly	Data visualization engine	Interactive, publication-quality SVG/WebGL charts (scatter, bars, pies, area curves) with dynamic tooltips.

7. How to Explain in a Technical Interview

When explaining this project to an interviewer, project reviewer, or hiring manager, structure your response in tiers: start with a high-impact elevator pitch, follow with the architectural workflow, and then demonstrate mastery over the engineering trade-offs.

30-Second Elevator Pitch (Memorize This):
"I built the **Codebase Evolution Intelligence System**—a software engineering analytics platform using Python, MySQL, Streamlit, GitPython, Pandas, Plotly, and Radon. While Git tracks file versions, it doesn't tell teams which files are accumulating technical debt. My system clones any public repository, mines its commit history, computes static McCabe Cyclomatic Complexity using Radon, and uses a multi-factor Hotspot algorithm to pinpoint high-maintenance files that combine frequent edits, recurring bug fixes, and high complexity. All data is persisted in a relational MySQL schema and visualized through an interactive Streamlit dashboard."

2-Minute Comprehensive Walkthrough Structure:

1. **Hook (Problem):** State that raw Git history contains critical patterns that are invisible without automated mining (e.g. churn, bug clustering, code complexity).
2. **Architectural Ingestion:** Explain that the system accepts a public GitHub URL, uses GitPython to clone it, and parses commit metadata, diff insertions, and deletions.
3. **Static Analysis with Radon:** Emphasize that you used Radon to parse Python Abstract Syntax Trees (ASTs) to compute McCabe Cyclomatic Complexity without executing arbitrary code.

4. **The Hotspot Formula:** Explain the multi-factor weighted formula (40% churn, 35% bug fixes, 25% complexity) to separate active configuration files from true architectural risks.
5. **Relational Persistence:** Describe the 6 MySQL tables, explaining why relational foreign keys and cascading deletes were critical for multi-repo analysis.
6. **Business Impact:** Conclude by showing how teams use this data for proactive refactoring, reviewing contributor distribution, and monitoring maintenance phases.

8. Top Interview Questions & Model Answers

Q1: Why did you design the Code Hotspot score using three specific dimensions?

"A single metric gives misleading results. A file with high cyclomatic complexity is harmless if developers rarely modify it. Conversely, a file with frequent commits might simply be a documentation or changelog file. A genuine architectural hotspot occurs at the intersection of three factors: high change frequency (40%), frequent bug-fix involvement (35%), and high cyclomatic complexity (25%). By normalizing these metrics across the repository and computing a weighted score from 0 to 100, the system surfaces the files that represent genuine technical debt."

Q2: Why did you label Bug-Fix Detection as a 'Heuristic'?

"In software engineering, commit message keyword detection (searching for words like 'fix', 'bug', 'error', 'hotfix') is an empirical heuristic, not a formal verification technique. Commit messages can have false positives (e.g., 'fixed styling indentation') or false negatives (e.g., 'address edge case in session cache'). Explicitly labeling this feature as a heuristic demonstrates professional software engineering rigor and realistic expectations about static data mining."

Q3: How does Radon calculate Cyclomatic Complexity, and why is it safe?

"Radon computes McCabe Cyclomatic Complexity by statically parsing Python code into an Abstract Syntax Tree (AST) using Python's standard `ast` library. It counts linearly independent paths through the code by calculating decision points (if-statements, for-loops, while-loops, comprehensions, and boolean operations). Because it only inspects the AST grammar nodes without executing the code, it is completely secure and safe against malicious repository code."

Q4: How did you solve the case-sensitivity collision issue in MySQL on Windows?

"In Git, file paths are strictly case-sensitive (e.g., `MANIFEST.in` vs `manifest.in` across commits). By default, MySQL collations like `utf8mb4_unicode_ci` are case-insensitive, which caused duplicate key violations on unique file path constraints. I resolved this by configuring `file_path VARCHAR(500) CHARACTER SET utf8mb4 COLLATE utf8mb4_bin NOT NULL` on the files table. The binary collation enforces byte-for-byte exact matching, aligning MySQL directly with Git's case-sensitive DAG semantics."

Q5: Why did you choose MySQL and SQLAlchemy over MongoDB or SQLite?

"Git repository history is deeply relational: a repository contains developers and commits; each commit modifies multiple files through a many-to-many relationship (`commit_files`); and files have associated `file_metrics`. A relational schema with foreign key constraints, indexes, and cascading deletes ensures referential integrity. When a repository is deleted or re-analyzed, cascading deletes automatically clean up all associated commits and file metrics. SQLAlchemy connection pooling also ensures connection reuse during high-volume operations."

Q6: How does the system handle high-contrast charts across dark and light OS themes?

"Streamlit applications inherit the user's operating system theme (Light or Dark Mode). Default Plotly charts can cause text labels to wash out against background canvases. I engineered a dedicated styling module (`chart_theme.py`) with `apply_chart_theme()`. It enforces crisp white card canvases (`#ffffff`), explicit deep-slate typography (`#0f172a`), subtle gridlines (`#e2e8f0`), and vibrant color palettes (electric blue, emerald, crimson, amber). This guarantees high visibility regardless of system theme."

Q7: What are the engineering limitations, and how would you scale this for enterprise use?

"Currently, the system clones repositories locally and analyzes commits up to a capped threshold (e.g. 2,500 commits). For enterprise scaling (e.g. repos with 100,000+ commits), I would: 1) Run cloning and parsing as distributed Celery/Redis background worker tasks, 2) Use shallow or blobless Git clones (`--filter=blob:none`) to fetch commit trees without downloading large binary histories, and 3) Implement multi-language complexity parsers (e.g. tree-sitter) for TypeScript, Go, and Java."

9. Key Takeaways for Technical Evaluators

The Codebase Evolution Intelligence System stands out in a technical portfolio because it goes far beyond simple CRUD operations. It demonstrates end-to-end full-stack engineering proficiency: low-level version control system inspection (GitPython), AST static code analysis (Radon), relational database design with connection pooling and cascading integrity (MySQL/SQLAlchemy), multi-dimensional data engineering (Pandas), and interactive data visualization (Streamlit/Plotly).